honey-nfc tag

https://gemini.google.com/app/bd163982ddea40dc

User prompt: I have to use Blockchain for this , but I have no idea on how to implement, how it functions and which to use etc . My web application is in bun and react , with supabase. This is my university project , i need to use all free tools. Also we will be using nfc tamper tags on the jar

Response: The Mental Model: What the Blockchain Actually Does Here

In supply-chain tracking, you **never store raw IoT sensor streams, images, or large JSON payloads directly on the blockchain**; doing so is slow, expensive, and unnecessary.

Instead, use a **State Anchor (Proof of Existence)** architecture:

1. **Supabase (PostgreSQL)** acts as your primary database storing hive weights, moisture tests, lab reports, and batch metadata.
2. When a batch is sealed or a lab report is approved, your backend takes that record, creates a cryptographic hash (SHA-256 or Keccak-256), and writes **only** the `batchId`, `dataHash`, `timestamp`, and `tamperStatus` to a Smart Contract.
3. When a consumer scans the NFC tag, the React app fetches the batch data from Supabase, recalculates its hash, reads the stored hash from the blockchain, and checks if they match. If anyone tampers with the database row, the hashes mismatch and the verification fails.

The Recommended 100% Free Stack

Layer	Tool	Cost	Why Use It
Network	**Polygon Amoy Testnet**	Free	EVM-compatible, fast, zero gas cost using free faucets, and provides a public block explorer (`amoy.polygonscan.com`) to show evaluators.
Contract IDE	**Remix IDE** (`remix.ethereum.org`)	Free	Browser-based, zero setup, compiles and deploys Solidity contracts with one click.
RPC / Node Provider	**Alchemy / Infura (Free Tier)** or Public RPC	Free	Connects your Bun backend and React client to the blockchain.
Blockchain Library	**`viem`** or **`ethers.js`** **(v6)**	Free	Works natively in Bun and React for reading/writing transactions.
Database & Auth	**Supabase**	Free tier	PostgreSQL with real-time updates and REST APIs out of the box.

1. The Smart Contract (Solidity)

Keep the contract focused strictly on batch provenance, audit trails, and seal states. Open Remix IDE, create `HoneyRegistry.sol`, and compile:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract HoneyRegistry {
    address public owner;

    struct Batch {
        string batchId;        // e.g. "HC-0421"
        bytes32 dataHash;      // Keccak256 hash of (origin, weight, labReportUrl, etc.)
        string nfcTagId;       // UID of the NTAG 424 DNA
        uint256 harvestTime;   // Unix timestamp
        bool isRecalled;       // FSSAI / Regulator recall flag
        bool exists;
    }

    mapping(string => Batch) public batches;
    mapping(string => string) public nfcToBatchId;
```

```
    event BatchRegistered(string indexed batchId, bytes32 dataHash, string nfcTagId);
    event BatchRecalled(string indexed batchId, string reason);

    modifier onlyOwner() {
        require(msg.sender == owner, "Unauthorized");
        _;
    }

    constructor() {
        owner = msg.sender;
    }

    function registerBatch(
        string calldata _batchId,
        bytes32 _dataHash,
        string calldata _nfcTagId,
        uint256 _harvestTime
    ) external onlyOwner {
        require(!batches[_batchId].exists, "Batch already registered");

        batches[_batchId] = Batch({
            batchId: _batchId,
            dataHash: _dataHash,
            nfcTagId: _nfcTagId,
            harvestTime: _harvestTime,
            isRecalled: false,
            exists: true
        });

        nfcToBatchId[_nfcTagId] = _batchId;

        emit BatchRegistered(_batchId, _dataHash, _nfcTagId);
    }

    function getBatch(string calldata _batchId) external view returns (Batch memory) {
        require(batches[_batchId].exists, "Batch does not exist");
        return batches[_batchId];
    }

    function flagRecall(string calldata _batchId, string calldata _reason) external onlyOwner {
        require(batches[_batchId].exists, "Batch does not exist");
        batches[_batchId].isRecalled = true;
        emit BatchRecalled(_batchId, _reason);
    }
}
```

Deploying for free:

1. Install MetaMask in your browser and switch to **Polygon Amoy Testnet**.
2. Claim free test tokens from the Polygon Faucet.
3. In Remix, select **Injected Provider - MetaMask** in the Deploy tab, click **Deploy**, and save the deployed contract address.

2. How the NFC Tamper Tag (NTAG 424 DNA) Connects

NTAG 424 DNA tags feature **SUN (Secure Unique NFC)** and a physical tamper wire loop (TagTamper).

1. **How it physically works:** The tag has a conductive loop across the jar lid. When sealed, the loop is intact (`TT status = 00`). When opened, the wire tears, permanently flipping the internal status bit (`TT status = 01`).
2. **Dynamic URL generation:** Every tap generates a URL with dynamic cryptographic query params:

   `https://honeychain.app/verify?uid=04A1B2C3D4E5F6&ctr=000005&cmac=E8B7A1F2C3D4E5F6&tt=00`
3. **Backend verification:**
   - Your Bun backend checks if the `cmac` matches the tag's AES secret key to prevent cloning.

- If `tt != 00`, the jar has been physically opened or tampered with.
- Your backend maps the `uid` to the database batch record.

3. Anchoring Data from Bun & Supabase to Polygon

Users and beekeepers should **never** be forced to pay gas fees or install MetaMask. Your Bun backend acts as the relayer, signing transactions using a dedicated server wallet funded by the faucet.

Install dependencies in Bun:

```
bun add viem @supabase/supabase-js dotenv
```

Anchor a batch from your backend (`anchor.ts`):

```
import { createWalletClient, http, defineChain, keccak256, toHex } from "viem";
import { privateKeyToAccount } from "viem/accounts";

// Minimal ABI of the contract function
const CONTRACT_ABI = [
  {
    name: "registerBatch",
    type: "function",
    stateMutability: "nonpayable",
    inputs: [
      { name: "_batchId", type: "string" },
      { name: "_dataHash", type: "bytes32" },
      { name: "_nfcTagId", type: "string" },
      { name: "_harvestTime", type: "uint256" }
    ],
    outputs: []
  }
] as const;

// Setup Polygon Amoy Chain
const polygonAmoy = defineChain({
  id: 80002,
  name: "Polygon Amoy",
  nativeCurrency: { name: "POL", symbol: "POL", decimals: 18 },
  rpcUrls: { default: { http: ["https://rpc-amoy.polygon.technology"] } }
});

const account = privateKeyToAccount(`0x${process.env.OPERATOR_PRIVATE_KEY}`);
const client = createWalletClient({
  account,
  chain: polygonAmoy,
  transport: http()
});

export async function anchorBatchToChain(batch: {
  id: string;
  nfcTagId: string;
  origin: string;
  moisture: number;
  harvestTime: number;
}) {
  // 1. Create a reproducible hash of the crucial off-chain data
  const dataPayload = JSON.stringify({
    id: batch.id,
    origin: batch.origin,
    moisture: batch.moisture,
    harvestTime: batch.harvestTime
  });
  const dataHash = keccak256(toHex(dataPayload));

  // 2. Commit the hash to Polygon Amoy
  const txHash = await client.writeContract({
    address: process.env.CONTRACT_ADDRESS as `0x${string}`,
    abi: CONTRACT_ABI,
```

```
    functionName: "registerBatch",
    args: [batch.id, dataHash, batch.nfcTagId, BigInt(batch.harvestTime)]
  });

  return { txHash, dataHash };
}
```

4. Zero-Friction Consumer Verification in React

When the consumer taps the jar, your React app verifies both the blockchain state and the tamper tag **without asking the consumer to connect a crypto wallet**.

Install dependencies in your React app:

```
bun add viem
```

Consumer verification view (`VerifyJar.tsx`):

```
import { useEffect, useState } from "react";
import { createPublicClient, http, keccak256, toHex } from "viem";

const CONTRACT_ABI = [
  {
    name: "getBatch",
    type: "function",
    stateMutability: "view",
    inputs: [{ name: "_batchId", type: "string" }],
    outputs: [
      {
        components: [
          { name: "batchId", type: "string" },
          { name: "dataHash", type: "bytes32" },
          { name: "nfcTagId", type: "string" },
          { name: "harvestTime", type: "uint256" },
          { name: "isRecalled", type: "bool" },
          { name: "exists", type: "bool" }
        ],
        type: "tuple"
      }
    ]
  }
] as const;

const publicClient = createPublicClient({
  transport: http("https://rpc-amoy.polygon.technology")
});

export default function VerifyJar() {
  const [status, setStatus] = useState<"verifying" | "valid" | "tampered">("verifying");
  const [details, setDetails] = useState<any>(null);

  useEffect(() => {
    async function verify() {
      // 1. Extract params from the URL provided by NFC tap
      const urlParams = new URLSearchParams(window.location.search);
      const ttStatus = urlParams.get("tt"); // "00" = intact, "01" = broken
      const tagUid = urlParams.get("uid");

      // Physical tamper loop check
      if (ttStatus && ttStatus !== "00") {
        setStatus("tampered");
        return;
      }

      // 2. Fetch record from your Supabase backend
      const res = await fetch(`/api/batch?tagUid=${tagUid}`);
      const dbRecord = await res.json();

      // 3. Re-calculate hash locally from database values
```

```
        const reconstructedPayload = JSON.stringify({
          id: dbRecord.id,
          origin: dbRecord.origin,
          moisture: dbRecord.moisture,
          harvestTime: dbRecord.harvestTime
        });
        const computedHash = keccak256(toHex(reconstructedPayload));

        // 4. Read source-of-truth hash directly from Polygon Amoy
        const onChainRecord = await publicClient.readContract({
          address: "0xYourDeployedContractAddress",
          abi: CONTRACT_ABI,
          functionName: "getBatch",
          args: [dbRecord.id]
        });

        // 5. Compare DB hash to On-chain hash
        if (onChainRecord.dataHash === computedHash && !onChainRecord.isRecalled) {
          setDetails(dbRecord);
          setStatus("valid");
        } else {
          setStatus("tampered");
        }
      }

      verify();
    }, []);

    return (
      <div style={{ padding: "2rem", fontFamily: "sans-serif" }}>
        {status === "verifying" && <p>Auditing cryptographic proof...</p>}
        {status === "valid" && (
          <div style={{ color: "green" }}>
            <h2> Authentic Honey</h2>
            <p>Batch: {details.id}</p>
            <p>Origin: {details.origin}</p>
            <p>Moisture: {details.moisture}%</p>
            <p>Physical Seal: Intact</p>
            <p>On-Chain Hash: Verified</p>
          </div>
        )}
        {status === "tampered" && (
          <div style={{ color: "red" }}>
            <h2> Tampered or Unverified Product</h2>
            <p>The NFC physical loop was broken or the batch hash does not match the ledger.</p>
          </div>
        )}
      </div>
    );
}
```

Step-by-Step Execution Plan

1. **Test the Contract**: Paste the Solidity code into Remix, connect MetaMask, get free POL from the Polygon faucet, and hit Deploy. Note the contract address.
2. **Wire Supabase**: In your `batches` table, add two columns: `blockchain_tx_hash` (TEXT) and `data_hash` (TEXT).
3. **Anchor from Bun**: Whenever a batch status transitions to `VERIFIED` or `PACKAGED`, trigger the `anchorBatchToChain()` function using Bun to sign the transaction.
4. **Demonstrate to Evaluators**:
    - Open the React app and tap/scan the test link. Show the green **"Authentic Honey"** status.
    - Open Supabase, manually modify the moisture value or origin in the database row, and refresh the consumer page.
    - Show that the app immediately flips to **"Tampered or Unverified Product"** because the recomputed hash no longer matches the immutable on-chain hash recorded on Polygonscan.

User prompt: For consumers (there are two types of user, farmers and consumers) , I was thinking of adding a qr code for them to check the status of jar. As each jar will be scanned in last stop that is before reaching or retailer. User might not have a phone with nfc reader

Response: Dual-tagging with both a **QR Code and an NFC tag** is standard in supply-chain traceability, and your prototype screens already anticipate printing QR labels alongside NFC seals. Because many budget smartphones lack NFC hardware, a QR code ensures universal consumer access.

However, a standard QR code is simply a printed link that anyone can photocopy and replicate onto counterfeit jars. To prevent this while keeping the solution 100% free, implement a **Two-Tiered Trust Architecture**.

1. Two-Tier Verification Model

Your React verification page should inspect the URL query parameters to determine whether the scan originated from a physical NFC chip or an external QR code, dynamically adjusting the trust badge:

Verification Factor	Tier 1: QR Code Scan	Tier 2: NFC Tap (NTAG 424 DNA)
User Requirement	Any camera app / browser	NFC-enabled smartphone
Origin & Hive Data	Verified via Blockchain hash	Verified via Blockchain hash
Lab Quality & Moisture	Verified via Supabase/Ledger	Verified via Supabase/Ledger
Retailer Shelf Hop	Checked against last-stop scan	Checked against last-stop scan
Physical Tamper Seal	⚠️ *Unchecked (Requires NFC)*	Cryptographically verified (Loop intact)

On your consumer results screen:

- **When opened via NFC:** All 4 indicators turn green: *Hive-verified*, *Lab report-verified*, *Store shelf-verified*, and *Jar sealed-verified*.
- **When opened via QR:** The first 3 turn green, while *Jar sealed* displays an amber warning: *"Tamper loop cannot be verified via camera. Tap with an NFC device or inspect physical seal intactness."*

2. Securing the QR Flow with the "Retailer Last Hop"

Your architecture specifies Step 10 ("Distribute - scan each hop") and the "Store shelf - verified" status. Use this retailer scan to validate the QR flow:

1. **Retailer Check-In:** When the retailer places the crate/jar on the shelf, they use the Beekeeper/Retailer portal to scan the jar.
2. **State Transition:** The backend marks the jar in Supabase as `status = ON_SHELF` with a timestamp and retail store ID.
3. **Scan Velocity & Replay Guard:**
    - A cloned QR code will be scanned repeatedly in different locations.
    - In Supabase, record a `scan_count` and log the IP/geolocation for every scan.
    - If a QR code marked `ON_SHELF` is scanned dozens of times or from disparate regions, the React interface flags a warning: *"Unusual scan activity detected for this jar. Possible duplicate label."*

3. URL Structure: Unified Verification Endpoint

Configure both the QR code and the NFC tag to hit the exact same React route, distinguished by URL parameters:

- **QR Code URL:**

  `https://honeychain.app/verify?batch=HC-0421&source=qr`

- **NFC Dynamic NTAG 424 DNA URL:**

  `https://honeychain.app/verify?batch=HC-0421&source=nfc&uid=04A1B2C3&ctr=0005&cmac=E8B7A1F2&tt=00`

4. Updating the React Verification Component

Update your verification logic to handle QR-only devices without throwing false "Tampered" errors:

```
import { useEffect, useState } from "react";
import { createPublicClient, http, keccak256, toHex } from "viem";

export default function VerifyJar() {
  const [loading, setLoading] = useState(true);
  const [data, setData] = useState<any>(null);
  const [sealState, setSealState] = useState<"verified" | "unchecked" | "broken">("unchecked");
  const [isTampered, setIsTampered] = useState(false);

  useEffect(() => {
    async function runAudit() {
      const params = new URLSearchParams(window.location.search);
      const batchId = params.get("batch");
      const source = params.get("source");
      const tt = params.get("tt"); // TagTamper loop status from NFC

      if (!batchId) {
        setIsTampered(true);
        setLoading(false);
        return;
      }

      // 1. Evaluate physical seal state
      if (source === "nfc") {
        setSealState(tt === "00" ? "verified" : "broken");
        if (tt !== "00") {
          setIsTampered(true);
        }
      } else {
        // Consumer scanned via camera QR
        setSealState("unchecked");
      }

      // 2. Fetch batch record from Supabase
      const res = await fetch(`/api/batch/${batchId}`);
      const record = await res.json();

      // 3. Re-hash database payload
      const localHash = keccak256(toHex(JSON.stringify({
        id: record.id,
        origin: record.origin,
        moisture: record.moisture,
        harvestTime: record.harvestTime
      })));

      // 4. Query Polygon Amoy testnet contract
      const client = createPublicClient({ transport: http("https://rpc-amoy.polygon.technology") });
      const onChainRecord = await client.readContract({
        address: "0xYourDeployedContractAddress",
        abi: CONTRACT_ABI,
        functionName: "getBatch",
        args: [batchId]
      });

      // 5. Audit data integrity and recall flags
      if (onChainRecord.dataHash !== localHash || onChainRecord.isRecalled) {
        setIsTampered(true);
      } else {
        setData(record);
      }
      setLoading(false);
    }

    runAudit();
  }, []);

  if (loading) return <p>Auditing batch provenance...</p>;
```

```
  if (isTampered) return <div style={{ color: "red" }}> Tampered or Unverified Product</div>;

  return (
    <div style={{ padding: "1.5rem" }}>
      <h2> Batch: {data.id}</h2>
      <p>Origin: {data.origin}</p>
      <p>Moisture Content: {data.moisture}%</p>

      <hr />
      <h3>Integrity Checklist</h3>
      <ul>
        <li> Hive Source: Verified</li>
        <li> Lab Report: Verified</li>
        <li> Store Shelf: {data.storeShelfVerified ? " Verified" : " Pending"}</li>
        <li>
          Physical Seal: {
            sealState === "verified" ? " Intact (NFC Hardware Validated)" :
            sealState === "broken" ? " Broken / Tampered" :
            "⚠️ Unchecked (QR scan cannot test wire loop; inspect seal manually)"
          }
        </li>
      </ul>
    </div>
  );
}
```

Low-Cost Physical Solution for Pure QR Jars

If a consumer uses QR and wants physical tamper-proofing without an NFC phone, add a **scratch-off 4-digit PIN** printed under a peel-back tamper sticker over the lid:

1. The QR directs the consumer to the batch page.
2. The user scratches off the one-time silver latex foil under the lid and inputs the 4-digit PIN.
3. Supabase marks that specific jar's PIN as `CONSUMED`. If an adulterer copied the QR label, the second customer to enter that PIN is immediately notified that the jar was previously unsealed.
